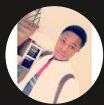


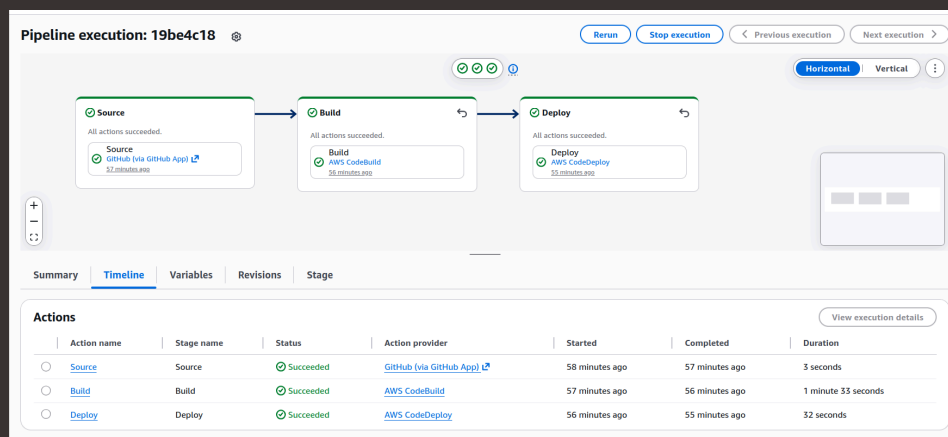


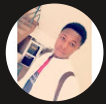
nextwork.org

Build a CI/CD Pipeline with AWS



Ahmed Tetteh





Introducing Today's Project!

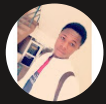
In this project, I will demonstrate CodePipeline's ability to automate my workflow. I'm doing this project to learn how CodePipeline can act as a central place to connect and oversee all my workflows, from building and running tests, to deployment. By the end of the project, I should be able to update the code for my webapp, push it and simply see all those updated changes in the live web app without manually going through CodeBuild and CodeDeploy.

Key tools and concepts

Services I used were AWS CodePipeline, CodeDeploy, CodeBuild, CodeArtifact, Amazon S3, GitHub, VS Code, IAM roles and policies, EC2 and CloudFormation. Key concepts I learnt include the different stages of the CI/CD pipeline, handling rollbacks and webhooks.

Project reflection

This project took me approximately 4 hours (including the startup and configuration of all other required resources, and error troubleshooting). The most challenging part was troubleshooting the errors that kept occurring during my deployment stage and rollback. It was most rewarding to see all the green marker indicators on all the stages of the pipeline, indicating a successful deployment of my application without going through the individual stages manually.



Ahmed Tetteh

NextWork Student

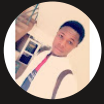
nextwork.org

Starting a CI/CD Pipeline

AWS CodePipeline is a tool that automates all our developer processes (CI/CD pipeline), i.e, from building and testing to deployment. It makes our workflow seamless by detecting changes in our code when pushed to our Github repository, triggers a build process in CodeBuild while running tests simultaneously (Continuous Integration), and then initiates a deployment of our application in CodeDeploy (Continuous Deployment). This significantly reduces our application deployment time for end-users.

CodePipeline offers different execution modes based on how you want to handle multiple runs of your pipeline. I chose Superseded execution mode due to the size and nature of my project, but other options include Queued and Parallel mode.

A service role gets created automatically during setup so that CodePipeline can have the necessary permissions to perform actions on your behalf, i.e, store the build artifacts in your S3 bucket, initiate a build with CodeBuild and deploy your update with CodeDeploy.



Developer Tools

AWS CodePipeline

Visualize and automate the different stages of your software release process

AWS CodePipeline is a continuous integration and continuous delivery service for fast and reliable application and infrastructure updates. CodePipeline builds, tests, and deploys your code every time there is a code change, based on the release process you define.

Create AWS CodePipeline pipeline

Get started with AWS CodePipeline by creating your first continuous delivery and continuous integration pipeline.

[Create pipeline](#)

Pricing (US)

Pay 12-type pipelines.
Each action execution* \$0.002/minute*
*100 free action execution minutes per month.
[*Learn more](#)

Getting started

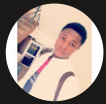
[What is AWS CodePipeline?](#)

[Setting up AWS CodePipeline](#)

[Working with AWS CodePipeline](#)

[Documentation](#)

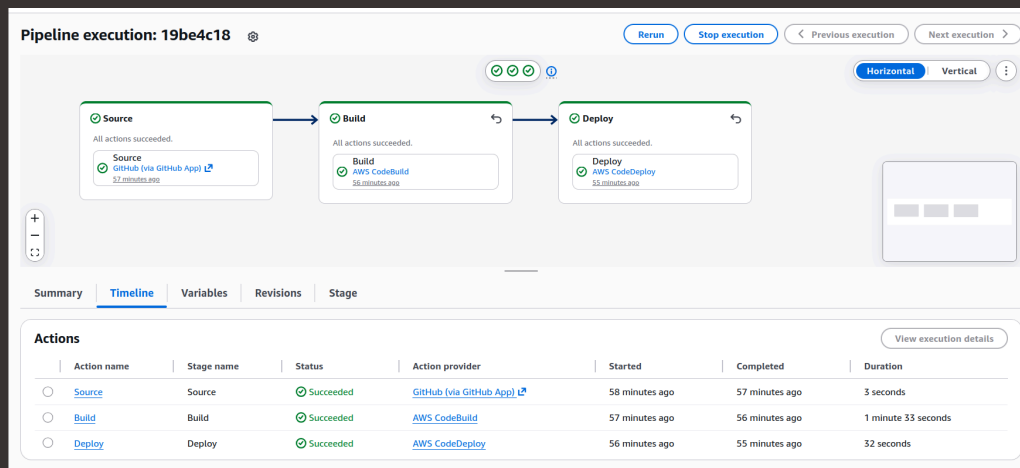
How it works

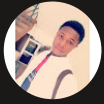


CI/CD Stages

The three stages I've set up in my CI/CD pipeline are Source, Build and Deploy. While setting up each part, I learnt about how CodePipeline connects to each them, and also does it job, i.e, from using the CodeConnection to fetch and package the source code into zipped file(source artifact) > pass that source artifact from the S3 bucket to CodeBuild > deploy the build artifact using deployment group in CodeDeploy.

CodePipeline organizes the three stages into a single flow diagram from Source to Deploy. In each stage, you can see more details on the pipeline execution by clicking on the Stage ID. This also reveals hyperlinks for each action provider or exact resource CodePipeline used (GitHub, CodeBuild and CodeDeploy).





Source Stage

In the Source stage, the default branch tells CodePipeline which branch in our repository to look out for changes or updates to our source code. So, it's simply telling CodePipeline the branch to use to start a pipeline execution.

The source stage is also where you enable webhook events, which is the tool that initiates the automation of my pipeline. It's like a notification for CodePipeline to start a pipeline execution. It basically waits and listens for changes in the main/master branch (i.e, webhook/link to the GitHub repo) of our source repository and triggers the automation of our pipeline.

Source

Source provider
This is where you stored your input artifacts for your pipeline. Choose the provider and then provide the connection details.

GitHub (via GitHub App) ▼

Connection
Choose an existing connection that you have already configured, or create a new one and then return to this task.

arn:aws:codestar-connection: X or [Connect to GitHub](#)

Repository name
Choose a repository in your GitHub account.

kingswanzy2020/nextwork-web-project X

You can type or paste the group path to any project that the provided credentials can access. Use the format 'group/subgroup/project'.

Default branch
Default branch will be used only when pipeline execution starts from a different source or manually started.

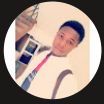
main X

Output artifact format
Choose the output artifact format.

☒ **CodePipeline default**
AWS CodePipeline uses the default zip format for artifacts in the pipeline. Does not include Git metadata about the repository.

☐ **Full clone**
AWS CodePipeline passes metadata about the repository that allows subsequent actions to do a full Git clone. Only supported for AWS CodeBuild actions. [Learn more](#)

☒ Enable automatic retry on stage failure



Build Stage

The Build stage sets up the connection to CodeBuild for our deployable. build artifact. I configured CodeBuild to use the source artifact as the input artifact of the build stage. The input artifact for the build stage is the compressed source code of the first stage in zipped file format.

The screenshot shows the 'Build - optional' configuration page in the AWS CodeBuild console. The 'Build provider' section has 'Other build providers' selected, with 'AWS CodeBuild' chosen from the dropdown. The 'Project name' field contains 'nextwork-devops-cicd', and the 'Define buildspec override - optional' checkbox is unchecked. The 'Environment variables - optional' section has an 'Add environment variable' button. The 'Build type' section has 'Single build' selected. The 'Region' dropdown is set to 'Asia Pacific (Tokyo)'. The 'Input artifacts' section has 'SourceArtifact' selected. The 'Enable automatic retry on stage failure' checkbox is checked.

Build - optional

Build provider
Choose the tool you want to use to run build commands and specify artifacts for your build action.

☐ Commands ☒ Other build providers

AWS CodeBuild

Project name
Choose a build project that you have already created in the AWS CodeBuild console. Or create a build project in the AWS CodeBuild console and then return to this task.

nextwork-devops-cicd or Create project

☐ Define buildspec override - optional
Buildspec file or definition that overrides the latest one defined in the build project, for this build only.

Environment variables - optional
Choose the key, value, and type for your CodeBuild environment variables. In the value field, you can reference variables generated by CodePipeline. [Learn more](#)

Add environment variable

Build type

☒ Single build
Triggers a single build.

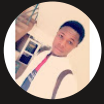
☐ Batch build
Triggers multiple builds as a single execution.

Region
Asia Pacific (Tokyo)

Input artifacts
Choose an input artifact for this action. [Learn more](#)

SourceArtifact
Defined by: Source

☒ Enable automatic retry on stage failure



Deploy Stage

The Deploy stage is where deploy the end user product of your application, in my case, using CodeDeploy. I configured the build artifact as the input artifact for my deployment, along with it's application and deployment settings.

Deploy - optional
Deploy provider
Choose how you want to deploy your application or content. Choose the provider, and then provide the configuration details for that provider.

AWS CodeDeploy

Region

Asia Pacific (Tokyo)

Input artifacts
Choose an input artifact for this action. [Learn more](#)

BuildArtifact ×
Defined by: Build
No more than 100 characters

Application name
Choose an application that you have already created in the AWS CodeDeploy console. Or create an application in the AWS CodeDeploy console and then return to this task.

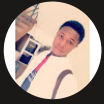
nextwork-devops-cicd ×

Deployment group
Choose a deployment group that you have already created in the AWS CodeDeploy console. Or create a deployment group in the AWS CodeDeploy console and then return to this task.

nextwork-devops-cicd-deploymentgroup ×

☒ Configure automatic rollback on stage failure
☐ Enable automatic retry on stage failure

[Cancel](#) [Previous](#) [Skip deploy stage](#) [Next](#)

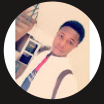


Success!

Since my CI/CD pipeline gets triggered by webhooks event whenever I push a change to the main branch of my GitHub repo, I tested my pipeline by making a change to the source code of my web app.

The moment I pushed the code change to GitHub, my pipeline was triggered by the webhooks event and did a new execution... The commit message under each stage reflects the latest trigger from GitHub for the pipeline execution. It also shows the latest change of the code.

Once my pipeline executed successfully, I checked my live web app to see verify the change on from the end-user perspective.



Ahmed Tetteh
NextWork Student

nextwork.org

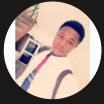
Hello King!

This is my NextWork web application working!

I am writing this line using nano instead of an IDE.

If you see this line in Github, that means your latest changes are getting pushed to your cloud repo :o

If you see this line, that means your latest changes are automatically deployed into production by CodePipeline!

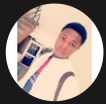


Testing the Pipeline

In a project extension, I initiated a rollback on the deploy stage to revert back to the previous version of my application, while maintaining the latest changes of my source and build stages. Automatic rollback is important for disaster recovery in cases where an update to the application has broken down some other features or introduced some bugs, hence, the application can be rolled back to the last known previous working version.

During the rollback, the source and build stage are unaffected by the current pipeline execution action because both stages come before the deployment stage for our rollback. I could verify this by comparing the commit messages related to each stage. I could see that both the Source and Build stage have the same commit message (i.e, they are using the same code change), while the Deploy stage has reverted back to the previous commit message.

After rollback, the live web app reverted to back to the previous state before the change in the code. This implies that, CodeDeploy was able to roll back the application to the previous version successfully, reducing application downtime and ensuring customer satisfaction.



Ahmed Tetteh
NextWork Student

nextwork.org

Deploy

Rollback

68a11c44-1f46-4c8f-86f3-002147a75db7

All actions succeeded.

Deploy

AWS CodeDeploy

2 minutes ago

07e69ee5 Source: Altered th



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

